

1 DMC tutorial - How to create a GUI plug-in

A central component for an efficient production is the support of the personnel through targeted visualization of work instructions, live data, etc. The requirements for comprehensive display options can be found in every production environment, but the specific needs are just as different as the products to be manufactured.

In order to meet this situation, DMC offers the options to design complex GUIs, which can be used flexibly thanks to consistent modularization.

This section describes the GUI concept implemented in DMC and the classes required for its use. The GuiPlugin sample, which is delivered with the DMC-SDK, is available to illustrate the described procedure.

Example

The GUI concept of DMC is based on the MVP principle (Model-View-Presenter). Presenters are implemented as full-fledged DMC components, giving full access to the option of the component system. The views can be designed independently with Windows Forms, Windows Presentation Foundation (WPF) or similar. The relevant user data such as events, work pieces, etc. are considered as the model.

There are four presenters and four views in the GuiPlugin sample: The MainPresenter displays the main window of the application (MainView). This includes three panels; the WorkstepPresenter, InstructionsPresenter and ButtonsPresenter, each with a corresponding view. The three panels are modular and have no dependencies among themselves.

Because each Windows GUI application requires a GUI thread with MessagePump, an ApplicationContext is generated in the StartComponent () method of the MainPresenter and the WinForms application is started. Similarly, a WPF application could be initialized and started.

Configuration

In accordance with the structure of the views, WorkstepPresenter, ButtonsPresenter etc. are configured as child components of the MainPresenter. The MainPresenter is assigned to a workplace:

```
<ComponentConfiguration Name="WorkstepPresenter"
    Class="Sample_GuiPlugin.WorkstepPresenter" />
<ComponentConfiguration Name="ButtonsPresenter"
    Class="Sample_GuiPlugin.ButtonsPresenter" />
<ComponentConfiguration Name="InstructionsPresenter"
    Class="Sample_GuiPlugin.InstructionsPresenter" />

<ComponentConfiguration Name="MainPresenter"
    Class="Sample_GuiPlugin.MainPresenter"
    Children="WorkstepPresenter,ButtonsPresenter,InstructionsPresenter" />

<ComponentConfiguration Name="PREHEAT"
    Class="mpdv.MachineDataCollector.Dmc.Entities.Workstation"
    Children="...,MainPresenter" />
```

Implementation

Each Presenter inherits from the PresenterComponent class. This represents an extension to the BaseEntity, but forwards events to and from its child components. Furthermore, the PresenterComponent implements INotifyPropertyChanged to inform the view of changes to its own status. The use of bindings is supported.

To include a component in a view, the GuiPlugin sample uses the PresenterComponentPanel. This uses the *Show ()* method of the PresenterComponent to display the presenter in a view. You can use data templates to unlink view and presenter under WPF. There is also nothing in the way to prevent the use of MVVM.